WHEN THE WINDOW CANNOT SEE THE QUESTION: TOKEN-DENOMINATED OBSERVATION WINDOWS ARE NOT LANGUAGE-NEUTRAL

Anonymous authors
Paper under double-blind review

ABSTRACT

Observation-window evictors such as SnapKV score the key–value cache using only the last w tokens of the prompt. We show that this integer is a language-dependent threshold. In the common `passages + question + instruction` layout, an instruction longer than w hides the question from the scorer: on Qwen3-4B the block of tokens after the question is 25 tokens in English but 107 in Bengali and 167 in Telugu, and the library default w=64 costs the latter two 15 and 20 points of accuracy on the full validation pools while leaving English untouched. The exposure is shipped, not hypothetical: moving to the constant of 32 that one serving path ships blinds six of our eight languages and costs Thai and Swahili 4.5 and 6.5 points on matched items, and in a survey of 22 popular templates frozen before reading, a shipped refine template leaves exactly 64 tokens after the query and an agent scaffold 221. The failure is not linguistic. Padding an English instruction reproduces it, a tokenizer that shortens the Bengali instruction moves the threshold with it, and an English prompt carrying a JSON response schema leaves a trailing block of 166 tokens — one from Telugu's 167 — and is blinded in exactly the same way. Recovery needs question-sized slack: a window four tokens past the instruction recovers nothing. We therefore set $w = c + Q_{90}$ from the tokenizer, the chat template and a held-out question-length percentile. Preregistered and at matched retained cache, this removes the blind mode wherever it appears — $+14$ pp over the default on the full 669-item Telugu pool ($p<10^{-17}$), $+13$ on Bengali — closes the schema tails, returns the shipped default where it already suffices, and meets its ± 3 pp gate on all five evaluated languages at n=100; on the full pool, Telugu instead certifies a -5.7 pp residual ($[-7.8, -3.1]$). The question-sized slack is load-bearing: on the same pool a 16-token slack leaves -10.3 pp, a preregistered head-to-head of $+4.6$ pp ($[+1.9, +7.0]$) in the percentile's favour. The same integers transfer unchanged to a second evictor with the same window ($+26$/$+31$ pp on its own baselines), and the residual is not a visibility failure: with the question moved last, fully visible by construction, eviction still costs -3.4 pp ($[-5.3, -1.2]$, n=669), and a per-item oracle window (V_i=1 exactly) recovers 0.6 pp of it ($[-1.2, +2.3]$).

1 INTRODUCTION

Serving a language model involves a surprising number of integers that count tokens: context limits, cache budgets, decode caps. Because tokenizers are not language-neutral (Petrov et al., 2023), each means something different in different languages; the consequences are documented for the budgets a user can see — cost, latency, usable context (Marchisio et al., 2024; Marie & Fujita, 2025). This paper is about a token-denominated integer that users cannot see. It sits inside the scoring rule of a widely used family of KV-cache eviction methods, and its default value decides whether the model is allowed to look at the question.

SnapKV (Li et al., 2024) and its descendants do not read the whole prompt when they decide what to evict: they score the cached keys against the last w tokens — the *observation window* — and keep the highest-scoring entries. kvpress ships w=64; the opt-in RocketKV path in TensorRT-LLM ships w=32. In the retrieval-QA layout `passages + question + instruction + chat`

`suffix`, that window is consumed from the right. Write c for the tokens standing after the question: if $c > w$, the window holds no token of the question, and the scorer decides what to keep without seeing what was asked (Figure 1).

On Qwen3-4B those trailing tokens — a one-line QA instruction plus the chat suffix — number 25 in English, 107 in Bengali and 167 in Telugu: at the shipped default, English prompts are safe while Bengali and Telugu are blind, losing 15 and 20 points on the full validation pools (Section 6). The mechanism, however, is not about those languages: padding the English instruction until c passes w reproduces the cliff without leaving English; a JSON response schema does the same and puts its trailing block one token from Telugu's; and a tokenizer that encodes the Bengali instruction in 26 tokens rather than 102 moves the cliff along with it. High-fertility tokenizers are the pathway that makes a fixed instruction overflow a fixed window, not a ranking of languages by difficulty.

The exposure turns on whichever constant is in force: moving between the two shipped constants on matched items flips Thai and Swahili across the threshold at a measured cost of 4.5 and 6.5 points (Section 4). And the layout is not our invention: among 22 popular templates surveyed blind, a shipped refine template leaves exactly the library default of 64 tokens after the query, and an agent scaffold leaves 221 — longer than any of our eight instructions (Appendix S).

Clearing the trailing block turns out to be necessary but far from sufficient: a window four tokens past c recovers nothing at all, and an English-sized slack of sixteen recovers most of the loss but not the last few points. What the scorer needs is enough *of the question*, which suggests measuring the quantity directly: query visibility, $V = |W \cap Q|/|Q|$ (Section 3), computable from the tokenizer and the template with no forward pass; Luo et al. (2026) showed that question visibility changes method rankings and left exactly such a continuous measure open.

The measurement then implies a rule that is computed rather than tuned: set the window to the trailing block plus a question-sized slack, $\hat{w} = c + Q_{90}$, where Q_{90} is a percentile of question length estimated on held-out questions. Both terms are measured rather than fitted to accuracy — the choice of the 90th percentile is a design decision, registered in advance, whose cost we report — no delimiter is required, and because SnapKV protects the window from within the retained budget, no extra cache is bought. Preregistered, the rule meets its ± 3 pp gate on English, Bengali and Telugu at Qwen3-4B (n=100); closes the English JSON-schema tails a mis-sized earlier version had broken by 23 points; and returns the shipped 64 where that constant is already adequate. On the full 669-item Telugu pool the gate does not hold: $\hat{w}$ leaves -5.7 pp ($[-7.8, -3.1]$) beside a $+14$ pp recovery over the default, joining the Bengali misses at Gemma-3-4b-it (-6), Qwen3-8B and Llama-3.1-8B (-8) — and we show the residual is not, in the main, a visibility failure: moving the question to the end of the prompt — full visibility by construction — still costs -3.4 pp ($[-5.3, -1.2]$) at the full pool, a per-item oracle window closes none of it, and the items the rule leaves broken are no poorer in fully visible questions than their pool (Fisher p=.62).

Contributions. (i) We localise a language-dependent failure of KV eviction to the relation between the observation window and the trailing block, with experiments that hold language fixed while c moves: an English instruction pad, a change of tokenizer, and an English response schema blinded one token from Telugu's block. (ii) We measure the dose-response, showing that the slack must be question-sized, and introduce query visibility V as an input-side, zero-inference measure of it. (iii) We propose and preregister AutoWindow, $\hat{w} = c + Q_{90}$, one integer per model, language and template, and report the gate, the intervals, and the misses. (iv) We diagnose the residual where the gate is missed and confirm the diagnosis at power: with the question last — fully visible by construction — the full Telugu pool still pays -3.4 pp ($[-5.3, -1.2]$), and a per-item oracle window (V_i=1 exactly) leaves -5.1 of $\hat{w}$'s -5.7. What remains is eviction cost, not blindness. (v) We show the integer belongs to the prompt, not the method: it transfers unchanged to a second evictor inheriting the same window, while at matched ratios a query-agnostic scorer loses in every language, English included, and one larger constant misses the Bengali gate the rule meets; neither outperforms the measured window anywhere we ran both. Appendix A maps every claim to its evidence and its interval status in one table. The broader offer is the discipline: a constant that counts tokens is a measurement in disguise, and the paper shows what auditing one costs and buys.

Scope. We study a method family rather than a shipped production default: vLLM exposes no SnapKV-style evictor, the RocketKV path is opt-in, and Mao et al. (2026) argue that research evic-

tors rarely ship as defaults (Appendix T). Our claim is narrower than harm to low-resource languages in general: a token-denominated window blinds whichever prompts carry a trailing block longer than itself, and under today's tokenizers such prompts are non-English — two of our eight languages at w=64, six at 32, English never. That last clause is why the number goes unaudited: no tested window costs English anything at this ratio (w=32 vs w=64: 2 pp, ns), so an English-validated benchmark cannot tell the two shipped constants apart; nothing in standard practice objects to whichever ships. Of the surveyed templates, most of what ships is safe — single-shot retrieval defaults leave 0–5 tokens after the question — and the long blocks concentrate where prompts are iterated (Appendix S).

2 Related work

Observation windows and query visibility. Prefill eviction scores the cache from a last-w slice of the prompt (Li et al., 2024). Luo et al. (2026) showed that whether the question falls inside that slice changes the *ranking* of compression methods on English long-context benchmarks, and left a continuous measure of query sensitivity open, noting that the natural candidates require a forward pass. V fills that slot from the input side: SnapKV's accumulated attention and the estimated attention of Devoto et al. (2025) are computed from the model, V from the tokenizer and the template alone. Expected Attention is the informative contrast: it removes the need to see the question by estimating future queries, and never reasons about a window at all.

How w gets chosen. Recent eviction work treats w as a hyperparameter to set rather than a threshold to test: IntentKV (Zhao et al., 2026) and the original SnapKV retrieval configuration use w=64, and Nawrot et al. (2026) sweep windows to a task-average optimum. None of it lets w fall below a trailing instruction whose length varies across otherwise matched items — the cell this paper occupies.

Instruction order, protection, and delimiters. Chen et al. (2026) document that eviction can discard an instruction in multi-instruction English prompts and propose a whitelist; their reordering moves two instructions inside a system prompt; we move the instruction relative to the question and grow it until it overflows the window, with a remedy that needs no knowledge of prompt structure. Garcia (2026) show that protecting a prompt's prefix and suffix outperforms scoring under a global cache cap, assuming the structurally critical tokens occupy a roughly fixed 15–30-token budget — while the Bengali and Telugu instructions occupy 102 and 162; we treat that assumption as a prediction and test it. Protection with a fixed suffix budget is, on our grid, the c+16 ablation under another name — protection with a measured budget is this paper's rule. FINCH (Corallo & Papotti, 2024) sizes its window from a user-supplied delimiter in a context–delimiter–question layout, so V=1 holds there by construction — evidence that w must adapt, via a delimiter and a question-last layout that the failure studied here does not enjoy.

Tokenizer fertility. Petrov et al. (2023) established that a fixed context budget is not language-neutral; subsequent work studied the cost, latency and usable context that follow, including under weight quantization (Marchisio et al., 2024; Marie & Fujita, 2025) — budgets the user can see and, in principle, plan around. The constant studied here sits inside the scorer, invisible from the outside; to our knowledge no published eviction study attributes per-language damage to it.

3 Setup

Task and data. Every item is a question, its gold passage, and same-language distractor passages packed to a fixed 8k-token prefill, with the gold passage rotating through front, middle and back positions by item index. Questions and passages come from XQuAD (Artetxe et al., 2020) for English, Chinese, Spanish, Vietnamese and Thai, and from TyDiQA-GoldP (Clark et al., 2020) for Swahili, Bengali and Telugu. The two collections are not comparable and XQuAD is parallel only within itself, so we never compare accuracy across languages. Every number in this paper is a within-language, item-paired difference between two configurations evaluated on the same items: 100 per cell, except where Section 6 pairs the entire validation pools (669 Telugu, 113 Bengali) and marked supporting runs pair 200.

Models and compression. Qwen3-4B (Qwen Team, 2025) is the primary model; Qwen3-8B is a scale slice and Gemma-3-4b-it (Gemma Team, Google DeepMind, 2025) a tokenizer contrast. We compress with SnapKV (Li et al., 2024) through the kvpress implementation at a fixed eviction ratio r=0.75 — three quarters of the prefill cache is discarded — and expose the observation window as the treatment variable. SnapKV keeps the window inside the retained budget (Appendix B), so every comparison below is at matched retained KV. Decoding is greedy with a 384-token cap: the more common 128-token cap is itself a token-denominated constant, and Appendix O reports what it did to us.

Scoring. An answer counts as correct if a gold span appears, after Unicode normalization and language-aware tokenization, either inside the marked answer span or inside the first sentence of the prose. No LLM judge is used anywhere in this paper. The rule was fixed before the runs reported here and is applied offline to raw generations; a stricter, marker-only variant leaves every conclusion it re-scores unchanged (Appendix D), with one qualification stated where it arises: the slack-ablation head-to-head keeps its ordering but not its individual significance under that scorer (Appendix E). Paired comparisons are two-sided McNemar tests, reported with their discordant counts (fixed/broken). Cells pair 100 items unless marked otherwise (the full-pool re-runs, and the n=200 supporting comparisons pairing the same 100 questions at two context budgets).

The quantities. Prompts are instruction-last unless stated: `passages + question + instruction + chat suffix`. Write I for the token length of the trailing instruction, s for the chat-template suffix that follows it, and $c{=}I{+}s$ for the number of tokens after the question. Write Q_i for the set of token positions holding item i's question, so that $|Q_i|$ is its length, and W for the set of positions in the last-w window. Query visibility is the share of the question the scorer can see, $V_i = |W \cap Q_i| \,/\, |Q_i| = \mathrm{clamp}((w-c)/|Q_i|, 0, 1)$ in this layout, where we take the intersection as the definition and the closed form as a corollary, so that interleaved layouts do not silently break it. On Qwen3-4B the eight instructions tokenize to I between 20 (English) and 162 (Telugu) tokens; with s=5 this gives c from 25 to 167 (Appendix F). Both c and Q are produced by script from the run's own tokenizer and template, never copied from a table (procedure: Appendix C; frozen instructions and exact layout: Appendix P).

The held-out percentile. Q_{90} is the 90th percentile of question token length on a split that excludes the 100 evaluation items (XQuAD validation after index 100; TyDiQA-GoldP train minus evaluation questions; n=1090/2387/5563 for en/bn/te), giving $18/76/80$ on Qwen3-4B. It is never estimated on scored items and never tuned against accuracy.

Software stacks. Each comparison stays inside one model and one software stack (driver, CUDA, torch, kvpress); cells from different stacks, models or tokenizers are never pooled, even when they carry the same configuration name. Follow-up runs re-ran cells that earlier stores already contained; across all 6,763 such generations the new text is byte-identical to the old (Appendix Q), which is what licenses reading the earlier and later tables as describing one system.

4 THE WINDOW IS A THRESHOLD

This section establishes three facts: damage tracks the window against the tail, not the language; it appears in English as soon as we lengthen the tail; and it appears at a constant already shipped.

A window sweep across five languages. Table 1 and Figure 2a report paired damage against the uncompressed baseline for five languages at five windows, ratio 0.75; the registered predictions were BLIND when $w < c$, SAFE when $w > c$, the step allowed to sit somewhat above c.

English never moves — not because every tested window holds its whole question (w=32 leaves seven tokens of slack against questions of 6–24) but empirically: partial visibility without damage recurs on Gemma (Appendix I). Bengali is damaged in every blind cell (-13 to -21 pp, all $p < .01$) and comes back to -3 pp only at w=176 (c+69). Telugu is damaged in all five cells, including w=176 — for Telugu that window is c+9 — where it is still -13 pp (p=.019, uncorrected; Appendix D); its -14-then--13 wiggle over the last two windows is one point at n=100, noise rather than structure. Thai and Swahili dip at the one tested window below their c and are within noise

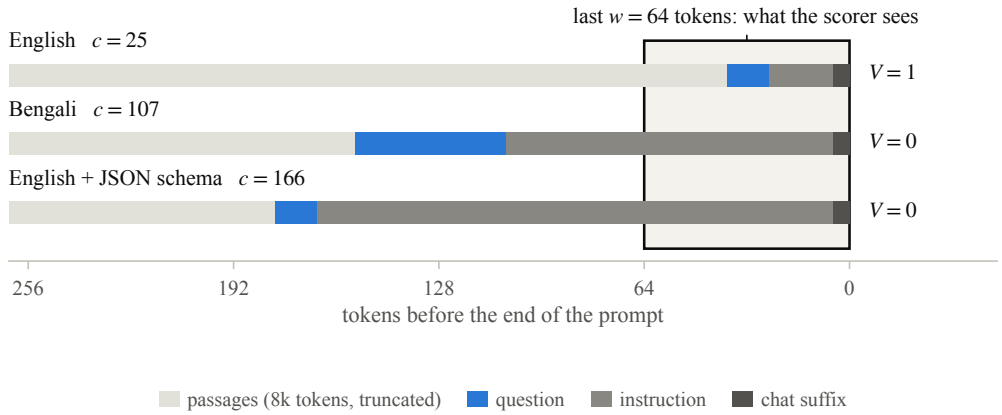

Figure 1: The instruction-last layout, drawn to token scale. Shaded: the last w=64 tokens, the only part of the prompt a SnapKV-style scorer reads. English leaves 25 tokens after the question, so the window covers it; Bengali leaves 107 and an English prompt carrying a JSON schema 166, and in both the scorer ranks the passages having seen none of the question. Widths are item medians; passages continue leftwards.

Table 1: Sweeping the observation window. Paired Δpp against the uncompressed baseline, Qwen3-4B, n=100 items per cell, eviction ratio 0.75. Shaded cells are those the registered rule called BLIND before the run, meaning $w < c$; stars mark McNemar $p < .05$. Shading is a property of the design and stars of the outcome, so the table is the comparison between the two. The w=64 column comes from the AutoWindow store (same stack, baselines byte-identical to this sweep's) and pairs within that store.

	c	baseline	w		default	w		
			32	56	64	88	120	176
en	25	93	+2	+1	+0	+1	−1	+1
th	45	85	−6	−4	+0	0	−2	−3
sw	47	56	−7*	−4	−3	−2	−4	−3
bn	107	73	−13*	−15*	−16*	−21*	−8	−3
te	167	56	−19*	−21*	−19*	−18*	−14*	−13*

elsewhere; we do not headline Swahili, whose block fits inside the default window, so it carries no default-window hole.

Two features matter more than the average effect. Damage follows the ordering of c, not of baseline accuracy: Swahili and Telugu share the lowest baseline (56%), and one loses at most 7 points at a single window while the other loses double digits everywhere. And clearing c is evidently not sufficient — Telugu at c+9 is still down 13 points — which is what Section 5 takes up.

The same cliff, constructed in English. If the mechanism is trailing length, padding an English instruction should reproduce the cliff and widening w past the new c should remove it. We pad the English QA instruction with content-free filler to four lengths, which put c at 57, 73, 105 and 137 tokens, and sweep five windows on the same English items (Figure 2b; Table 8).

The threshold moves with the pad: in each condition accuracy peaks at the first tested window exceeding that condition's c and at no earlier one (the bold cells in Table 8); at the sharpest case, w=104 — one token short of its c=105 — scores 90% where w=144 scores 93%. Plotted against the slack $w - c$ the four conditions collapse onto one curve (Figure 2b): every window short of its condition's trailing block scores at most 90%, every window clearing it at least 91% — a separation we read as a description of the grid, not a test, since neighbouring cells sit within 100-item noise; across these curves only the number of tokens between the question and the end of the prompt changed.

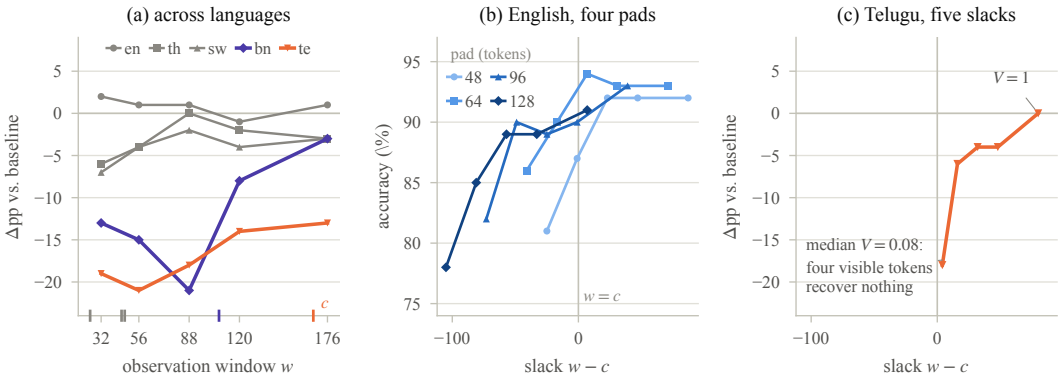

Figure 2: Three sweeps of the same mechanism, each paired against its own uncompressed baseline. **(a)** Five languages against the window; ticks mark each language's c. **(b)** English, four instruction pads, against the slack $w - c$: the conditions collapse onto one curve, separated at $w{=}c$. **(c)** Telugu, same x-axis: English is at its ceiling within about ten tokens of slack, Telugu needs eighty.

A constant that is already shipped. kvpress ships $w{=}64$; the opt-in RocketKV path in TensorRT-LLM ships $w{=}32$. Moving from the first constant to the second, on matched items, costs Thai 4.5 pp (2/11, $p{=}.022$) and Swahili 6.5 pp (1/14, $p{=}.001$; $n{=}200$ — the same 100 questions at two context budgets, so we lean on the discordant counts; store `w32`). Both languages sit above the threshold at $w{=}64$ and below it at $w{=}32$: the failure is not confined to the two scripts with the longest instructions but reaches any language whose trailing block outgrows whichever constant is in force. Thai and Swahili would take $\hat{w} = 91$ and 67, both above their own c: the rule cannot produce this flip. A preregistered arm later measured it: both languages meet the ± 3 pp closure gate at their own $\hat{w}$ — Thai -1 pp, non-inferior by interval; Swahili -3 pp at the boundary, interval wide (Appendix E). A third handle points the same way: moving the instruction to the front of the prompt — the question inside every window, press and budget untouched — recovers Bengali by 13 pp and Telugu by 14 pp on the same items (both $p{<}.02$; Table 2) — a lever on the mechanism, not a deployment recommendation.

5 The slack must be question-sized

How far past the trailing block must the window reach? Telugu at $c{+}9$ was still 13 points down, so "clear c" is not the answer.

A dose ladder. We sweep five windows at fixed offsets above the measured c on Telugu, the language with the longest trailing block and the deepest hole: $c{+}4$, $c{+}16$, $c{+}32$, $c{+}48$ and $c{+}80$, on 100 items with their own uncompressed baseline (Figure 2c; Table 9 in Appendix G; offsets fixed before the run, c re-measured on the run tokenizer).

Four visible tokens of question buy nothing: $c{+}4$ loses 18 pp — what the same items lose at the shipped $w{=}64$, seeing none of the question. The curve then rises with the slack and reaches the baseline only at $c{+}80$. The same ladder on Bengali, with its own c and baseline, reproduces it rung for rung (-17, -7, -3, -5, -2 pp), including a $c{+}4$ rung as useless there as here (Appendix G).

Visibility as the dose. Question length varies across items (34–122 tokens), so one window already spreads visibility across items, and each item passes through several values of V as the window widens. Pooling the five treated cells and binning by per-item V: items with $V < 0.25$ lose 16.1 pp ($n{=}118$, $p{<}10^{-3}$), while items with $V{=}1$ show no detectable effect ($+0.7$ pp, $n{=}135$; 9/8 discordant). Across the ladder, a logistic regression of correctness on V with item-clustered standard errors gives $+0.68$ ($z{=}3.35$, $p{<}.001$); within items, 21 switchers are correct at high visibility against 2 in the opposite direction ($p{<}10^{-4}$). On treated rows, V is also the narrowly better one-parameter summary: AIC prefers $y \sim V$ (689.8) to $y \sim (w{-}c)$ (692.1) and to $y \sim (w{-}c) + |Q|$ (692.2) — a gap of two is support, not a discrimination.

Table 2: AutoWindow at eviction ratio 0.75. Accuracy, with paired Δpp against each block's own uncompressed baseline in parentheses; n=100; star: McNemar $p < .05$. (A) Qwen3-4B on three of the eight languages; (B) English prompts carrying a JSON schema tail, c measured with the schema in place. The last column is the layout remedy — the question moved to the end of the prompt, V=1 by construction — against that layout's own baseline. The full-pool re-run is reported below and in Appendix E.

	window		accuracy (Δpp vs. baseline)			layout
	c	$\hat{w}$	uncompressed	w=64	at $\hat{w}$	Δpp
(A) languages						
en	25	43	93	93 (+0)	**95** (+2)	95 (−1)
bn	107	183	73	57 (−16*)	**71** (−2)	70 (−4)
te	167	247	56	37 (−19*)	**56** (+0)	51 (−6)
(B) English with a JSON schema tail						
JSON-60	72	90	94	88 (−6)	**96** (+2)	—
JSON-120	166	184	95	89 (−6*)	**96** (+1)	—
JSON-200	213	231	94	89 (−5)	**96** (+2)	—

The bins are not monotone throughout, and we state where. The $0.75 \leq V < 1$ band loses 11.9 pp (n=59, p=.065), and the Bengali ladder breaks in the same band; both hold long-question items that recover once V reaches 1 (Appendix G). We claim the window-level dose curve and the within-item trend, not a monotone per-item bin profile.

Slack therefore counts in units of the question, not constant tokens: an English-question-sized slack leaves the median Telugu item at V=0.31.

6 AutoWindow: one integer, and where it stops

The rule. Both quantities Sections 4 and 5 identify are available before any forward pass: c from the tokenizer and chat template, a question-sized slack from held-out questions. We set $\hat{w} = c+Q_{90}$: one integer per (model, language, template), computed offline, no delimiter. A per-item window w_i=$c + |Q_i|$ would give V_i=1 by construction, and where a serving path can locate the question in every request it is the natural refinement — but a preregistered full-pool run of that oracle buys almost nothing (the residual paragraph below), and per-item sizing puts a question-locating parser on the serving path, the assumption FINCH's delimiter buys its way out of and we decline (Corallo & Papotti, 2024) (Appendix C). The language-level percentile is a deliberately weaker commitment: $\hat{w}$ covers the entire question for $98/113$ Bengali and $611/669$ Telugu items; the remaining decile is registered leftover, not a claim.

The registered gate. Panel (A) of Table 2 is the preregistered test ($|\Delta| \leq 3$ pp against the uncompressed baseline). The formula meets it at n=100, and gains +14 and +19 pp on Bengali and Telugu against the shipped default (16 fixed / 2 broken and 21/2; $p \leq .001$). The English-sized ablation c+16 behaves as Section 5 predicts: it recovers most of the hole and stops short of the baseline (−7 and −6 pp; the direct c+16-vs-$\hat{w}$ comparison is settled at the full pool below). Across the eight-language grid, the languages the default window does not blind stay within 3 pp of baseline bar one marginal exception (Appendix F): the gate is a statement about the languages it does. Re-run on the entire validation pools (the preregistered everything-there-is rule, Appendix E), Bengali holds the gate (−1.8, n=113; its capped pool adds no discordant item beyond the original hundred) with a +13.3 pp recovery (18/3, p=.0015); Telugu does not: −5.7 pp (19/57, CI $[-7.8, -3.1]$), against a −20.2 pp default hole ($[-22.4, -17.3]$) and a +14.5 pp recovery (p=7×10^{-18}). The original hundred reproduce byte-identically and still sit at zero; the 569 new items sit at −6.7 — the n=100 gate was sampling luck, and the full-pool number is the headline.

The percentile, separated from a constant slack. Is Q_{90} doing work a fixed slack would not? A preregistered ablation re-ran the entire Telugu pool at c+16, c+32 and $\hat{w}$ in one store with its own baseline (byte-identical to the depth arm's on all 1,338 shared generations). Every window below $\hat{w}$ leaves a strictly larger certified residual (−10.3 at c+16, −7.8 at c+32, against −5.7), and the

registered head-to-head separates the rule from the English-sized slack: $+4.6$ pp over $c{+}16$ (55/24, $p{=}.0006$, CI $[+1.9, +7.0]$). Over $c{+}32$ the point still favours $\hat{w}$ but the interval includes zero ($+2.1$, $[-0.3, +4.2]$); under the marker-only scorer the ordering holds but the $c{+}16$ head-to-head alone is ns (Appendix E); it replaces an earlier unregistered $n{=}100$ pooling.

Schema tails, and a construction we got wrong first. The same logic applies when the tail is a response format. An earlier run sized the window from the *unpadded* English instruction ($w{=}41$, below the very default it meant to improve on) and cost 23 pp at the 120-token schema. Measuring c with the schema in place gives 72, 166 and 213 tokens, hence $\hat{w} = 90$, 184 and 231. Panel (B) shows the same formula closing all three tails while breaking none of the items the default window answered correctly (8/0, 7/0, 7/0) — by interval, the best-supported close in the paper (Appendix E). The 120-token schema's trailing block, 166 tokens, sits one token from Telugu's 167: the same blindness, produced by a response format instead of a script, removed by the same integer (both runs: Appendix H).

A second tokenizer. Gemma-3-4b-it tokenizes the same Bengali instruction to 26 tokens against Qwen's 102 and its questions to a median of 11, so both inputs move: $c{=}27/32/44$ and $Q_{90}{=}18/18/20$ for en/bn/te, giving $\hat{w} = 45/50/64$. For Telugu the rule returns exactly the constant kvpress already ships, and Telugu meets the gate there (-3 pp; CI $[-8.6, +4.2]$): a window rule worth running should also be able to say when the shipped constant already passes. For Bengali, $\hat{w}{=}50$ leaves -6 pp, which misses the same ± 3 pp criterion the 4B cells met — and the default leaves -5: one plateau, on which the rule neither digs nor fills a hole; its value here is saying correctly that there is no blind mode to remove (Appendix I).

Scale, prefill, and a third family. A Qwen3-8B slice reproduces the phenomenon and not the gate: the default costs Bengali 17 pp, $\hat{w}$ recovers $+9$ and remains 8 below baseline; English is at ceiling (Appendix J). Llama-3.1-8B-Instruct re-measures both inputs on a third tokenizer family ($\hat{w}{=}212$ for Bengali): the hole reproduces at -18 pp ($[-21.5, -9.2]$), the formula recovers $+10$ and leaves -8; we do not claim it closes on either model (Appendix K). At Qwen3-32B the Bengali hole persists, certified (-12.0 $[-13.9, -4.5]$), $\hat{w}$ misses the gate at -5, and the Telugu cell cannot distinguish a hole from noise at $n{=}100$ ($[-12.4, +5.6]$) — scale softens the magnitudes (Appendix L). At twice the prefill nothing moves: 16k Telugu loses -15.0 ($[-19.7, -5.7]$) at the default, and the SAME $\hat{w}{=}247$ — its inputs are prefill-free — meets the gate with a $+13.0$ recovery ($p{=}.0023$).

What the residuals are not. Four cells miss the closure gate: Bengali at Gemma (-6), 8B and Llama (-8), and Telugu at the full pool (-5.7, certified). The diagnosis was made after seeing the first three; the fourth, at $6.7\times$ the sample, obeys it. With the question last every window sees all of it by construction, and a preregistered instruction-first re-run of the entire Telugu pool certifies that eviction still costs -3.4 pp there ($p{=}.0027$, CI $[-5.3, -1.2]$) — within the registered ± 3 pp of the $\hat{w}$ residual — with no concentration by gold position or question length (Fisher $p \geq .17$). A preregistered per-item oracle window — $V_i{=}1$ exactly, no percentile leftover — leaves -5.1 ($[-7.3, -2.4]$), indistinguishable from $\hat{w}$ ($+0.6$, $[-1.2, +2.3]$); its residual is depleted at the front gold position (10.7% vs 33.3%, $p{=}.0001$) and concentrated mid-document — the fingerprint of retrieval-difficulty eviction cost, which visibility, constant across positions, cannot produce. The missed cells agree in proportion-test form: each residual is as rich in fully visible questions as its pool (89% vs 91% at the full Telugu pool, 8/8 vs 88% at 8B, 10/11 vs 88% at Llama, all $p \geq .59$), and at Gemma the gap is a plateau from 48 to 64 (Appendices I–K, E). Widening the window turns off the blind mode; it does not remove the ordinary cost of discarding three quarters of the cache.

Nor is the integer a property of one method: dropped unchanged into PyramidKV (Cai et al., 2024), which inherits the same window, it takes Bengali from -28 to -2 and Telugu from -32 to -1 (own baselines; Appendix N).

7 Analysis

Pooling the window sweep (3,000 generations, 500 items) and regressing correctness on compression and blindness ($\mathbf{1}[w < c]$) indicators, errors clustered by item, gives $+0.09$ log-odds for compression when the question is visible ($[-0.05, 0.23]$) and -1.06 for blindness ($z{=}-9.4$; uncom-

pressed rows are never blind, so the interaction equals blindness). We cannot distinguish the compression cost from zero when the scorer sees the question — not a demonstration that it is zero: the interval admits a small effect either way, and excludes anything resembling the blindness term. (AIC model comparison: Appendix G.)

Results that do not support the account. A stuffed arithmetic-reasoning control does not reproduce the effect (Bengali -3 pp, ns): the failure needs a question matched against passages, not merely a long prompt. Two scorers that never look at the question (Devoto et al., 2025; Oren et al., 2024) fail differently and worse, costing even English 14 and 8 pp where no window costs it anything (Appendix N). Early-stack pilots of 4-bit KV quantization damaged languages in a non-fertility order (not re-run on the campaign stack); a capacity mechanism exists in a different regime — at aggressive eviction even long-passage English items fail — and at this ratio we found no Bengali damage gradient across gold-length or retention strata (not a certified absence); we do not use high-dose fertility slopes as evidence about the window.

8 Limitations

The failure needs something long standing between the question and the end of the prompt; a deployment that puts the question last has $V{=}1$ and nothing here applies. The tail must also be long in the deployment's own terms — one *English* instruction serving Bengali and Telugu questions ($c{=}25$) produced no significant damage at the default window (-3/-4 pp, ns; Appendix M) — and even a fully hiding tail is not sufficient: the shipped refine layout, blind for every item, shows no certified damage at $n{=}100$ (Appendix S).

Two properties of the remedy are by design, not oversight: the percentile leaves the longest question decile below $V{=}1$, and because SnapKV protects the window it scores from, a very large w at an aggressive ratio taxes the content budget — small here, measured at $r{=}0.9375$ (Appendix N). Deploying the rule assumes the prompt family is known: one integer per trailing-block variant, so mixed traffic needs per-family measurement or the maximum over families — the mis-sized schema run is what skipping it looks like. On the surveyed templates that maximum is the agent scaffold's $c{=}221$, $\hat{w}{=}239$ in English; Appendix N's $w{=}256$ row prices a window that size — at $r{=}0.9375$ it protects half the retained cache and costs English nine points. For a deployer the guidance splits: sizing the window removes the blind mode wherever the default digs one ($+9$ to $+19$ pp at $n{=}100$, $+13$/$+14$ at the full pools); the residual is a property of eviction at that ratio, which no window — oracle included — removes: the lever left is retaining more cache (Appendix N).

Our items are constructed rather than drawn from a standard suite — a deliberate trade: published suites vary question length, instruction length and language together, exactly the confound we must hold fixed, at the price of leaderboard comparability. The headline cells now rest on the full validation pools (669 Telugu, 113 Bengali) — enough to certify the Telugu hole, its residual, the slack separation and the instruction-first residual (Appendix E), while Bengali's capped pool leaves its interval wide ($[-8.0, +5.2]$); other cells rest on 100 items, with closure intervals of roughly $\pm$7–9 pp (Appendix E). Finally, one primary 4B model carries the headline result; the 8B and 32B slices share its tokenizer — replications in scale, not independent samples, and at 32B the effects soften to where $n{=}100$ leaves Telugu inconclusive — and one contrast tokenizer cannot map the space of tokenizers.

9 Conclusion

A token-denominated observation window is not language-neutral, because trailing instructions are not. Two quantities, both computable before any inference, decide the outcome: the trailing block c and the slack beyond it — too little is blindness; too much, at heavy eviction, a tax. Setting $w = c + Q_{90}$ removes the blind mode, certifiably outperforms an English-sized slack at scale, leaves a certified residual that no oracle visibility removes, and transfers unchanged to a second evictor: the integer belongs to the prompt, not the method. The lesson generalises — our own decode cap chose which scoring branch judged each language (Appendix O): derive the constants that count tokens; do not ship them.

ETHICS STATEMENT

This work uses publicly released question-answering corpora and publicly released model weights; it involves no human subjects and no personal data. The failure it documents falls unevenly on users who write in scripts that current tokenizers encode inefficiently, so we have tried to describe it in terms of the mechanism rather than in terms of language rankings, and to avoid comparisons of absolute accuracy between languages that our item construction does not support. The remedy we propose is cheap and requires no additional data collection.

REPRODUCIBILITY STATEMENT

Configurations, seeds, prompts and the scoring rule are in the supplementary code. The two constants are produced by scripts, not copied between documents: `measure_c.py` (and `measure_c_schema.py` for schema tails) computes c from the run tokenizer and chat template, and `measure_q.py` computes Q_{90} from a held-out question split. The dose-response readout is produced by `v_trace_bins.py`, committed before the corresponding data existed, and the eight last arms' readouts by `iclr9_readout.py` and `iclr10_readout.py` against preregistered readings. Twenty-nine generation stores back the main arms and their controls — `cliff_multi`, `cliff_en`, `autowin`, `autowin_q90`, `autowin_8b`, `cliff_gemma`, `gemma_q90`, `schema`, `schema_fix`, `v_trace`, `v_trace_bn`, `llama`, `instr_first`, `agnostic`, `ratio`, `pyramidkv`, `constant`, `depth`, `constant_depth`, `slack_depth`, `if_depth`, `thsw`, `xinstr`, `refine`, `oracle_depth`, `ctx16k`, `qwen32b`, `w32` and `stuffed` — and Appendix O draws on two further cap-128-era stores, `e3` and `e3_384`. Every table cell in this paper is recomputed from the raw generations in those thirty-one stores by `audit_paper_numbers.py`, which fails on any drift, and comparisons never cross a stack boundary (Appendix Q). The predictions and kill conditions for every arm are files that ship with the code, each written before its run; for eighteen of the twenty-one arms the version-history timestamp also precedes the first generation in the corresponding store, while the other three — the Q_{90} remeasurement, the 8B slice and the schema re-run — entered version control in a later batch. For the constant-at-scale arm, the public anonymised release carrying its registration was pushed 48 minutes before its first generation, so a publicly visible external timestamp precedes every row it governs. For the eight fourth- and fifth-review arms the registrations were pushed to the hosting server before the first generation — over nine hours for the former, over an hour for the latter; third-party server timestamps that precede every row — but to the private working repository: the public anonymised release that carries them was rebuilt only after their results existed, and we state that distinction rather than blur it. The anonymized version history ships with the supplementary material, and the de-anonymized history becomes public with the code on publication.

AI USE STATEMENT

Large language models were used substantially and throughout this work, and this statement follows the categories of the ICLR AI policy. Research ideation, experimental design and the preregistered predictions were developed by the author in dialogue with LLMs; every preregistration was reviewed, approved and committed by the author before the corresponding run. Experiments were executed by LLM agents operating the experiment harness against those committed protocols, whose kill conditions bound what an agent could decide alone. The harness, the analysis scripts and the figures are code written with substantial LLM assistance; the paper's text was drafted and revised the same way. Interpretation of results was constrained by design rather than delegated: no LLM judges any output in any reported metric, every table cell is recomputed from the raw generation records by a committed audit script, and cross-checking between independent models during preparation surfaced and corrected errors, including fabricated bibliography entries and two internal miscounts, before submission. LLMs were not used to generate synthetic evaluation data; all items derive from XQuAD and TyDiQA-GoldP. The author reviewed all AI-assisted content and takes full responsibility for every claim in this paper.

REFERENCES

Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. On the cross-lingual transferability of monolingual representations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4623–4637, 2020. doi: 10.18653/v1/2020.acl-main.421.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

Alex Chen, Renato Geh, Aditya Grover, Guy Van Den Broeck, and Daniel Mingyi Israel. The pitfalls of KV cache compression. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 41530–41553, 2026. doi: 10.18653/v1/2026.acl-long.1926. ACL Anthology 2026.acl-long.1926.

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470, 2020. doi: 10.1162/tacl_a_00317.

Giulio Corallo and Paolo Papotti. FINCH: Prompt-guided key-value cache compression for large language models. *Transactions of the Association for Computational Linguistics*, 12:1517–1532, 2024. doi: 10.1162/tacl_a_00716. Anthology 2024.tacl-1.83.

Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025.

Gabriel Garcia. Protection is (nearly) all you need: Structural protection dominates scoring in globally capped KV eviction. *arXiv preprint arXiv:2605.18053*, 2026.

Gemma Team, Google DeepMind. Gemma 3 technical report, 2025. arXiv:2503.19786.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024.

Daming Luo, Christy Liang, and Junyu Xuan. How query visibility changes KV-cache compression rankings: A matched-budget audit. *arXiv preprint arXiv:2607.11942*, 2026.

Weian Mao, Yukang Chen, Wei Huang, Shuai Yang, Luozhou Wang, and Song Han. KV cache compression and its infra problems. NVIDIA Efficient AI Lab research blog, 12 June 2026, 2026. Cited against a serving-default claim.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. How does quantization affect multilingual LLMs? *arXiv preprint arXiv:2407.03211*, 2024.

Benjamin Marie and Atsushi Fujita. The uneven impact of post-training quantization in machine translation. *arXiv preprint arXiv:2508.20893*, 2025.

Piotr Nawrot, Jianing Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo Ponti. The sparse frontier: Sparse attention trade-offs in transformer LLMs. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 38667–38701, 2026. doi: 10.18653/v1/2026.findings-acl.1926. ACL Anthology 2026.findings-acl.1926; arXiv:2504.17768.

Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state RNNs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18724–18741, 2024.

Aleksandar Petrov, Emanuele La Malfa, Philip Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, 2023.

Qwen Team. Qwen3 technical report, 2025. arXiv:2505.09388.

Liang Zhao, Xiaocheng Feng, Weihong Zhong, Lei Huang, Kun Zhu, Baoxin Wang, Dayong Wu, Guoping Hu, Ting Liu, and Bing Qin. Question tells you where the answer is: Intention-aware long-context KV cache compression. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27153–27169, 2026. doi: 10.18653/v1/2026.acl-long.1250. ACL Anthology 2026.acl-long.1250.

A EVERY CLAIM, ITS EVIDENCE, AND ITS STATUS

Table 3 maps each claim the paper makes to the cell that carries it and to its status in the closure vocabulary of Appendix E. Reading guide: *certified* means the exact 95% interval clears the relevant bound; *confirmed* that it excludes zero; *gate met, wide* that the registered ± 3 pp point rule passed but $n{=}100$ cannot bound the interval; *ns* that no effect was detected at the stated power. Nothing in this table is claimed beyond its status column.

Table 3: Claim-by-claim evidence map. "Full pool" = the entire TyDiQA validation pool (669 Telugu / 113 Bengali).

claim	evidence	status
default $w{=}64$ blinds bn/te; holes at scale	full pools, App. E	certified (te $[-22.4, -17.3]$; bn -15, 3/20)
the cliff is reproduced in English by padding	pad grid, Fig. 2b	design-level grid, $n{=}100$/cell
an English JSON schema blinds identically	Tab. 2B	certified close after resizing (3 tails non-inferior)
the threshold moves with the tokenizer	Gemma sweep, App. I	consistent sweep, $n{=}100$
$\hat{w}$ recovers the holes vs the default	full pools, App. E	certified ($+14.5$, $p{=}7{\times}10^{-18}$; $+13.3$, $p{=}.0015$)
closure gate at $\hat{w}$, five languages, $n{=}100$	Tab. 5	met on all five; non-inferior en+th; others wide
Q_{90} slack beats $c{+}16$ at scale	slack ablation, §6	certified ($+4.6$ $[+1.9, +7.0]$); vs $c{+}32$ ns
Telugu residual at $\hat{w}$, full pool	App. E	certified miss (-5.7 $[-7.8, -3.1]$)
the residual is not a visibility failure	IF + oracle full pools	confirmed (-3.4; oracle -5.1, h2h $+0.6$ ns)
the integer transfers to PyramidKV	App. N	significant ($+26$/$+31$, $n{=}100$)
shipped-constant flip costs th/sw	§4, $n{=}200$	significant ($p{=}.022$/.001)
an English instruction removes the damage	App. M	ns at $n{=}100$ (registered branch)
query-agnostic scorers lose everywhere	App. N	significant, English included
the per-item oracle window beats $\hat{w}$	oracle arm, App. E	refuted at $n{=}669$ ($+0.6$, $[-1.2, +2.3]$)
the same integer works at twice the prefill	16k slice, App. L	certified hole; gate met, recovery $p{=}.0023$
the hole persists at 32B	App. L	bn certified (-12); te inconclusive at $n{=}100$
the shipped refine layout is damaged	App. S	ns (en -3; bn -8, $p{=}.057$) — geometry only

B HOW LAST-w SCORING WORKS

SnapKV forms $q_{\text{obs}} = q[:, :, -w\ :]$, scores the prefix keys against that slice, keeps the highest-scoring entries under the compression ratio, and protects the window itself from eviction. The kvpress implementation defaults to $w{=}64$ with a pooling kernel of size 5, and the RocketKV path skips sparse KV entirely when the sequence is shorter than its prompt budget. Because the window

is retained rather than added, raising w at a fixed compression ratio does not increase the amount of cache that survives: it changes which entries are selected, not how many.

C MEASURING c AND Q_{90}

c is produced by `measure_c.py`. A probe prompt is wrapped in the model's chat template with `add_generation_prompt`; if the probe question's token ids appear as a contiguous subsequence, c is the number of tokens after that span (`after_question`); otherwise $c = I + s$, where s is the assistant-header suffix measured on a minimal turn (`fallback_I_plus_suffix`). Qwen3-4B takes the fallback path with s=5 and Gemma-3-4b-it the direct path with s=6. We verified the fallback against a direct offset-mapping measurement on real prompts: for all eight Qwen languages the two agree within one token, the exception being a single token of Thai where the boundary falls inside a merged token.

The instruction's second sentence exists only to make scoring deterministic, so `measure_c.py --no-marker` also reports c without it, cutting at the last sentence-ending punctuation before the marker: $14/16/19/21$ for English, Chinese, Spanish and Vietnamese, 26 for Swahili, 69 for Bengali and 98 for Telugu. Thai joins its two clauses with a conjunction and no punctuation, so it has no separable format sentence and we do not cut it on a guess. The headline cells do not rest on that clause: a minimal one-sentence instruction still leaves Bengali at 69 and Telugu at 98, past the default window, and under minimal instructions the blind set at w=32 still holds Bengali and Telugu — and Thai, whose instruction has no separable sentence to cut, remains blind there under its full c=45 either way.

For a padded or schema-carrying instruction, `measure_c_schema.py` builds the instruction with the same padding routine the item builder uses and reports c for the padded string, so the measured constant and the generated prompt cannot drift apart.

Q_{90} is produced by `measure_q.py` as the 90th percentile of `len(tokenizer.encode(question))` on a split that excludes the 100 evaluation items: XQuAD validation after index 100, and TyDiQA-GoldP train with every evaluation question id removed. The evaluation items themselves are never used to estimate it.

Two TyDiQA-GoldP raw-split warts are disclosed rather than silently patched. Bengali's raw train and validation splits share three exact duplicate examples; all three sit inside `validation[:100]` and are therefore removed from the Q_{90} source by the evaluation-id exclusion above. Swahili's splits share two, at validation positions 133 and 300 — outside the n=100 evaluation set, so they remain in the estimation pool; recomputing $Q_{90,\mathrm{sw}}$ with them removed leaves the percentile unchanged (20, n=2753 vs 2755). In every arm the invariant actually relied on holds and is asserted by the driver before generating: no evaluated item contributes to any Q_{90} source (for Swahili the evaluated set is `validation[:100]`). The guard that found the Swahili pair aborted a pod run before any generation existed; the log and the registered amendment ship with the record.

D THE SCORING RULE AND ITS ROBUSTNESS

An answer counts as correct if, after NFKC normalization, lowercasing, punctuation stripping and language-aware tokenization (character-level for scripts written without word separators; article stripping only for languages that have articles), some gold span appears inside the marked answer span or inside the first sentence of the prose that precedes the marker. Whole-output containment would over-credit answers that merely quote a passage; marker-only scoring under-credits models that state the answer in prose and then emit a reformatted token after the marker, and that failure rate differs by language, which would manufacture exactly the kind of per-language gap this paper is about. An earlier frozen rule recorded during generation was marker-only; we never quote it and recompute every cell offline.

The choice does not carry the results. Under a stricter marker-only rule the Telugu dose ladder still shows the same shape: the uncompressed baseline scores 50% rather than 56%, c+4 scores 35% and c+Q_{90} scores 47%, so the hole is -15 pp instead of -18 pp and the c+Q_{90} cell is -3 pp instead of 0 pp. Applied to the full-pool Telugu re-run of Appendix E, the same rule leaves the certified

residual in place: -9.4 pp (25 fixed / 88 broken, $p{=}2.1{\times}10^{-9}$, interval $[-11.9, -6.5]$). Table 4 repeats the 8B Bengali cells under two further scorers.

Table 1 reports 25 paired tests. Under a Holm correction across the grid, five blind cells survive — Bengali at $w{=}56$ and 88, Telugu at $w{=}32$, 56 and 88 — and the single-window Thai and Swahili dips, Bengali at $w{=}32$ ($p{=}.0072$, Holm threshold .0025), and the Telugu $w{=}120/176$ cells do not. No claim in the paper rests on an unsurviving cell alone: the Telugu $c{+}\varepsilon$ point is carried by the dose ladder, whose $c{+}4$ cell stands at $p{=}.0009$ in a six-test family.

Table 4: Robustness of the 8B Bengali cells to the scoring rule. The spelling fold maps two pairs of Bengali graphemes that the model interchanges; gold-token recall accepts a prediction covering at least 70% of the gold tokens. The ranking and the sign of every comparison are unchanged.

scorer	baseline	$w{=}64$	$\hat{w}$	$\hat{w}-$ baseline
containment (used throughout)	81	64	73	-8
containment after a spelling fold	81	65	75	-6
gold-token recall ≥ 0.7	85	70	79	-6

E CLOSURE, WITH INTERVAL ESTIMATES

The registered closure rule — $|\Delta| \leq 3$ pp against the uncompressed baseline, as a point estimate — is a decision rule fixed before each run, not an equivalence test, and at $n{=}100$ the two are far apart. Table 5 reports an exact 95% interval for every closure cell, obtained by conditioning on the discordant count N: given N, the number of items the treatment fixes is binomial, and a Clopper–Pearson interval for its rate maps monotonically onto an interval for Δ. Six cells support the stronger reading that the lower bound excludes a loss beyond -3 pp: English at 4B and at Llama, Thai at its own $\hat{w}$, and all three schema tails. The Bengali and Telugu intervals at 4B are consistent with the gate they met but also with losses the gate would have rejected — which is what $n{=}100$ can and cannot say — and the same holds for the Gemma, Llama and instruction-first cells. The 8B interval excludes zero, confirming that residual. The recovery claims against the default window ($+9$ to $+19$ pp, $p{\leq}.022$) are unaffected: they are significance claims and carry their own tests.

Two of those cells were then re-run at depth. The stopping rule was fixed before any of it existed: final n is the entire validation pool, 669 Telugu and 113 Bengali items, a rule with no free parameter and therefore nothing to stop early on. Telugu's interval now lies entirely below the gate rather than merely below zero — the stronger reading the table calls a certified residual — and the decomposition is exact: the original hundred items reproduce byte-identically and still sit at 0.0, while the 569 new ones sit at -6.7. Telugu's 57 residual items are no poorer in fully visible questions than the pool: 51 of 57 (89%) have $|Q| \leq Q_{90}{=}80$ against 611 of 669 (91%) in the pool (Fisher $p{=}.62$; $|Q|$ recomputed from the tokenizer by the audit) — the visibility diagnosis Section 6 quotes, in test form. Bengali's thirteen items beyond the original hundred contain no discordant pair at all: its full-pool row is the $n{=}100$ row over a larger denominator, not an independent confirmation — the pool is simply exhausted. Bengali meets the gate on a pool its own size caps at 113, so its interval stays wide, as the preregistration said it would. The 8B Bengali cell is consequently no longer the only residual in this paper that an interval confirms; the primary model now carries one too, measured at $6.7\times$ the sample.

Three later preregistered arms extend the table. The slack ablation re-ran the full Telugu pool at $c{+}16$, $c{+}32$ and $\hat{w}$ in one self-contained store (baselines byte-identical to the depth arm's on all 1,338 shared generations): each fixed slack leaves a certified residual strictly deeper than $\hat{w}$'s, and the registered head-to-head gives $\hat{w}$ $+4.6$ pp over $c{+}16$ (55/24, $p{=}.0006$, $[+1.9, +7.0]$) and $+2.1$ over $c{+}32$ ($[-0.3, +4.2]$, ns). Under the marker-only scorer the ordering repeats (43.9/45.1/45.6 against a 55.0 baseline) with the $c{+}16$ head-to-head at $+1.6$ ($[-1.1, +4.3]$), directionally consistent and individually not significant. The instruction-first arm re-ran the same pool with the question last: the confirmed -3.4 residual in the table is what eviction costs when visibility cannot be the cause, and its broken items show no concentration by gold position or question length (Fisher $p \geq .17$; marker-only scoring is uninformative in this layout, which suppresses the marker itself — the Appendix R behaviour). The Thai and Swahili rows are the closure gate at the two languages the rule had never been run on; both meet it, Thai with a non-inferior interval, Swahili at the point boundary with the

interval wide — so the rule's evaluated set is five of the eight languages, and no new miss joins the residual table.

Table 5: Exact-conditional 95% intervals for the paired difference at $\hat{w}$; `closure_cis.py`. The instruction-first rows are mechanism-gate cells — the default window with the question last, so $V{=}1$ by construction — judged by the same rule.

cell	n	Δpp	fixed/broken	95% CI	reading
4B en	100	$+2$	2/0	$[-1.4, +2.0]$	non-inferior at -3 pp
4B bn	100	-2	6/8	$[-9.1, +5.9]$	interval too wide
4B te	100	0	6/6	$[-6.9, +6.9]$	interval too wide
4B th	100	-1	1/2	$[-2.9, +2.4]$	non-inferior at -3 pp
4B sw	100	-3	2/5	$[-6.5, +2.9]$	gate met; interval wide
full pool: te	669	-5.7	19/57	$[-7.8, -3.1]$	certified residual
full pool: bn	113	-1.8	6/8	$[-8.0, +5.2]$	gate met; interval wide
full pool: te $c{+}16$	669	-10.3	24/93	$[-12.7, -7.4]$	certified residual
full pool: te $c{+}32$	669	-7.8	18/70	$[-9.8, -5.2]$	certified residual
full pool: te IF at $w{=}64$	669	-3.4	16/39	$[-5.3, -1.2]$	confirmed residual
full pool: te oracle w_i	669	-5.1	22/56	$[-7.3, -2.4]$	confirmed residual
te @16k, $w{=}64$	100	-15	3/18	$[-19.7, -5.7]$	certified residual
te @16k, $\hat{w}$	100	-2	3/5	$[-6.6, +4.1]$	gate met; interval wide
32B bn	100	-5	2/7	$[-8.5, +1.8]$	gate missed; interval wide
32B te	100	-1	2/3	$[-4.5, +3.5]$	gate met; interval wide
refine en at $w{=}64$	100	-3	1/4	$[-4.9, +2.2]$	interval too wide
refine bn at $w{=}64$	100	-8	3/11	$[-12.7, +0.2]$	interval too wide
JSON-60	100	$+2$	3/1	$[-2.4, +3.9]$	non-inferior at -3 pp
JSON-120	100	$+1$	1/0	$[-0.9, +1.0]$	non-inferior at -3 pp
JSON-200	100	$+2$	2/0	$[-1.4, +2.0]$	non-inferior at -3 pp
Gemma bn	100	-6	3/9	$[-10.7, +1.7]$	interval too wide
Gemma te	100	-3	4/7	$[-8.6, +4.2]$	interval too wide
8B bn	100	-8	0/8	$[-8.0, -2.1]$	confirmed residual
Llama en	100	$+1$	1/0	$[-0.9, +1.0]$	non-inferior at -3 pp
Llama bn	100	-8	3/11	$[-12.7, +0.2]$	interval too wide
Llama te	100	-1	3/4	$[-5.6, +4.4]$	interval too wide
IF bn at $w{=}64$	100	-4	3/7	$[-8.7, +3.0]$	interval too wide
IF te at $w{=}64$	100	-6	1/7	$[-7.9, +0.4]$	interval too wide

F Full window, pad, and $c{+}16$ grids

Table 6 gives the measured constants for all eight languages. Table 7 gives the eight-language run of the $c{+}16$ ablation: the five languages whose trailing block already fits inside the default window stay within 3 pp of baseline at both settings, with a marginal Vietnamese exception at the default window; only Bengali and Telugu carry a hole for $c{+}16$ to close, and it does not close it. Table 8 gives the English pad sweep plotted in Figure 2b.

The Vietnamese exception is worth pinning down, because Vietnamese is the one safe-set language whose cell brushes the gate (-5 at $w{=}64$). Its held-out question percentiles on this tokenizer are $Q_{50}/Q_{90}/Q_{\max} = 17/26/46$ ($n{=}1090$), and with $c{=}39$ the default window leaves 25 tokens of slack — so only 3 of the 100 evaluation items have $V < 1$ at all. The five items the default window breaks all have $V{=}1$: none of the damage sits on the partially visible items, so the -5 is ordinary compression residual of the kind Section 6 diagnoses, not a shallow copy of the Bengali–Telugu blind mode. The rule would set $\hat{w}_{\mathrm{vi}} = 65$, one token above the shipped default.

G The dose ladder in full

The Telugu ladder is the upper block of Table 9. Binning its five treated cells by per-item visibility gives: $V < 0.25$, $n{=}118$, -16.1 pp (6 fixed / 25 broken, $p{=}9 \times 10^{-4}$); $0.25 \le V < 0.5$, $n{=}104$,

Table 6: Measured constants, Qwen3-4B. I is the instruction length, $c{=}I{+}s$ with $s{=}5$, and Q_{90} is the held-out question-length percentile ($n{=}1090/2387/5563$ for en/bn/te; Q_{50} is $12/46/54$ and $Q_{\max}$ is $36/141/170$). Q_{90} was measured for the languages the AutoWindow arms use and for the two languages the $w{=}32$ flip touches in the sweep; Spanish and Vietnamese also have $c > 32$ but are not swept.

	en	zh	es	vi	th	sw	bn	te
I	20	24	30	34	40	42	102	162
c	25	29	35	39	45	47	107	167
$c{+}16$	41	45	51	55	61	63	123	183
Q_{90}	18	—	—	—	46	20	76	80
$\hat{w} = c{+}Q_{90}$	43	—	—	—	91	67	183	247

Table 7: The $c{+}16$ ablation across eight languages. Accuracy, with paired Δpp against the uncompressed baseline in parentheses; $n{=}100$; star: McNemar $p < .05$. Shading marks the blind cells, as in Table 1: of the eight languages, the shipped window blinds two.

	c	$c{+}16$	baseline	$w{=}64$	$c{+}16$
en	25	41	93	93 (+0)	94 (+1)
zh	29	45	83	83 (+0)	85 (+2)
es	35	51	88	86 (−2)	88 (+0)
vi	39	55	83	78 (−5)	81 (−2)
th	45	61	85	85 (+0)	84 (−1)
sw	47	63	56	53 (−3)	56 (+0)
bn	107	123	73	57 (−16*)	66 (−7)
te	167	183	56	37 (−19*)	50 (−6)

-4.8 pp; $0.5 \le V < 0.75$, $n{=}84$, -2.4 pp; $0.75 \le V < 1$, $n{=}59$, -11.9 pp (2/9, $p{=}.065$); $V{=}1$, $n{=}135$, $+0.7$ pp (9/8). Because an item can revisit a bin at more than one rung, the n here count observations, not items: the $V{<}0.25$ bin holds 118 observations of 100 distinct items and the $V{=}1$ bin 135 of 93. Recomputed with each distinct item counted once (its mean treated correctness in the bin against its baseline), the two ends move to -18.0 and 0.0 pp — the multiplicity flattered the blind end slightly and the conclusion is unchanged.

The $0.75 \le V < 1$ band is the one that breaks monotonicity, so we describe its contents. Its 59 observations are 59 distinct items — each item enters the band at exactly one rung, since visibility rises with the window — and they arrive through three of the five: 14 at $c{+}32$, 40 at $c{+}48$ and 5 at $c{+}80$. The damage is carried by nine of them, with questions of 40 to 81 tokens; six of those nine break at $c{+}48$, and four are answered correctly once the window reaches $V{=}1$. The band is not significant on its own ($p{=}.065$) and it represents 59 of the 500 treated observations. Read together with the $V{=}1$ bin, it says that visibility close to 1 is not yet equivalent to visibility at 1 for long questions — which is the same statement as the registered leftover of a language-level percentile, seen per item.

The Bengali ladder, the lower block, was run on its own grid with the same readout script and repeats the shape. Its bins are $V < 0.25$, $n{=}129$, -14.0 pp (5 fixed / 23 broken, $p{=}9 \times 10^{-4}$); $0.25 \le V < 0.5$, $n{=}95$, -7.4 pp; $0.5 \le V < 0.75$, $n{=}70$, -5.7 pp; $0.75 \le V < 1$, $n{=}59$, -8.5 pp (2/7, ns); $V{=}1$, $n{=}147$, no detectable effect (0.0 pp, 9/9). The same per-item dedup applied here moves the two ends to -15.0 (129 observations, 100 items) and -0.4 pp (147 observations, 88 items). Both ends land where the account says they should, and the band that breaks monotonicity is the same one, which makes the caveat above a two-language observation rather than an artefact of one grid. Across the ladder the cluster-robust logit on V gives $+0.65$ ($z{=}2.95$, $p{=}.003$); within items, 18 switchers are correct at high visibility against 1 in the opposite direction ($p{=}10^{-4}$), and AIC prefers V (637.7) to $(w{-}c)$ (639.7) and to $(w{-}c) + |Q|$ (641.5) — again a gap of two, which is support rather than a discrimination. An earlier run of this block was cut short at 43 items on its third rung and tied on that last comparison; the tie resolves, narrowly, once the ladder is complete. Its numbers are superseded here, and the 343 generations the two runs share are byte-identical.

Table 8: English pad sweep. Accuracy, n=100 per cell, eviction ratio 0.75. This store carries no uncompressed baseline, so the comparison is across windows; the bold cell in each row is the first window exceeding that pad's c, and it is also that row's maximum.

pad	c	w=32	56	80	104	144
48	57	81	87	**92**	92	92
64	73	86	90	**94**	93	93
96	105	82	90	89	90	**93**
128	137	78	85	89	89	**91**

Table 9: The dose ladder. n=100 per rung, ratio 0.75, paired against each language's own baseline; median V is the per-item share of the question inside the window. The top rung of each block is that language's $\hat{w} = c{+}Q_{90}$.

w	slack $w{-}c$	median V	accuracy	Δpp	fixed/broken	p
Telugu, c=167, baseline 56						
171	4	0.08	38	-18^*	5/23	.0009
183	16	0.31	50	-6	5/11	.21
199	32	0.62	52	-4	4/8	.39
215	48	0.92	52	-4	6/10	.45
247	80	1.00	56	0	6/6	1.0
Bengali, c=107, baseline 73						
111	4	0.09	56	-17^*	3/20	.0005
123	16	0.34	66	-7	5/12	.14
139	32	0.68	70	-3	4/7	.55
155	48	1.00	68	-5	3/8	.23
183	76	1.00	71	-2	6/8	.79

One-parameter descriptions of the sweep. We also compared one-parameter descriptions of the compressed rows, and the comparison does not fully favour us. AIC prefers language fixed effects (2744) over the distance to the threshold $(w{-}c)$ (3046), the binary blindness indicator (3075), and the raw window w (3220). The mechanism's prediction that $(w{-}c)$ beats raw w holds. But on a five-language grid $(w{-}c)$ is nearly collinear with language identity, and the sweep alone therefore cannot separate a threshold from a per-language constant.

H SCHEMA TAILS: THE MIS-SIZED RUN AND THE RERUN

The first schema run applied the AutoWindow rule but computed c from the *unpadded* English instruction, giving w=41 — smaller than the default w=64 it was supposed to improve on. It therefore tested a window below the threshold rather than the rule, and the result was correspondingly bad: against uncompressed baselines of $94/95/94$ for the 60-, 120- and 200-token schema tails, the default window scored $88/89/89$ (-6, -6^*, -5 pp) and the mis-sized window scored $88/72/75$ (-6, $-\mathbf{23}^*$, $-\mathbf{19}^*$ pp). The mis-sizing is itself an instance of the paper's claim: an English-derived constant applied to a prompt whose trailing block is much longer.

The rerun measures c with the schema in place ($72/166/213$) and is reported in panel (B) of Table 2. The two runs share their uncompressed and default-window cells, which the rerun reproduced exactly, generation for generation.

I A SECOND TOKENIZER: GEMMA-3-4B-IT

Gemma-3-4b-it encodes the same instructions much more compactly than Qwen3-4B: c is $27/32/44$ for English, Bengali and Telugu against Qwen's $25/107/167$, and its questions are shorter in tokens as well ($Q_{90} = 18/18/20$ against $18/76/80$). The threshold should therefore move down with the tokenizer, and the window sweep in Table 10 is consistent with that: Bengali is damaged at the two windows below its c and drifts to a non-significant -5 pp at the two above it, while Telugu — whose

Qwen counterpart loses 19 points at w=64 — shows no comparable hole anywhere on the grid. English shows no detected damage anywhere on the grid.

Table 10: Gemma-3-4b-it window sweep. Paired Δpp against the Gemma baseline (accuracy 75/62/37 for en/bn/te), n=100, star: McNemar $p < .05$. Shading marks blind cells as in Table 1. Not pooled with any Qwen cell.

	c	w=16	24	32	48	64
en	27	+2	+1	+3	+4	+3
bn	32	−13*	−10*	−9*	−5	−5
te	44	−3	−4	−3	−1	−3

Three qualifications belong with this table. Bengali at w=32, which is c exactly — still total blindness by the V formula, since the window holds only the trailing block — loses 9 pp, consistent with its blind neighbours; the registered shading rule $w < c$ leaves that boundary cell unshaded, a one-token conservatism we note rather than repaint; Telugu's baseline of 37% is low enough that the sweep would not resolve a small hole; and English is itself blind at w=16 and 24 on this tokenizer without being damaged there — indeed no window we tested damages Gemma's English at this ratio. Blindness is necessary for the failure but not sufficient. How much any ranking is worth is measured in Appendix N: evicting at the same ratio with no ranking at all collapses generation itself, so even a blinded ranking does most of the work. What distinguishes Gemma's undamaged blind English cells from Qwen's damaged padded ones, that measurement cannot say, and we report the asymmetry as an observation rather than an account. The within-model contrast is untouched — the same two blind windows cost Bengali 13 and 10 points — but the shading makes the asymmetry visible, and we would rather state it than let a reader find it. The AutoWindow arm on this tokenizer gives $\hat{w} = 45/50/64$ and returns $+3/-6/-3$ pp against baseline, against $+3/-5/-3$ at the default window. For Telugu the rule selects the default constant itself. For Bengali it does not close, and the main text explains why we read that residual as a plateau rather than a threshold effect: it is present at every window from 48 upward, and at $\hat{w}$=50 the median Bengali question of 11 tokens is entirely inside the window — 7 of the 9 items it breaks are fully visible, against 86% of the pool (Fisher p=.61).

J SCALE: QWEN3-8B

The scale slice covers English and Bengali with the same formula and the same 100 items per cell. English is at ceiling and carries no information: 96 baseline, 97 at the default window, 96 at $\hat{w}$=43. Bengali reproduces the phenomenon: 81 baseline, 64 at the default window (−17 pp, 1 fixed / 18 broken, p<.001), and 73 at $\hat{w}$=183, which recovers +9 pp against the default window (11/2, p=.022) but stays 8 pp below baseline (0/8, p=.008). The registered criterion was ±3 pp, so this is a miss, and we report it as one.

What the residual is not is worth stating precisely, because it is the same diagnosis that applies to Gemma. Of the eight items that the baseline answers and $\hat{w}$ does not, all eight have questions shorter than Q_{90} (38 to 67 tokens), so the window covers the question entirely for every one of them — against 88% fully covered in the pool (Fisher p=.59): the residual is, if anything, richer in visible questions than the items it is drawn from, not poorer, which is the direction a visibility failure cannot produce. Seven of the eight reproduce the question verbatim in the generated text; and all twelve items whose question exceeds Q_{90} — the only items where the language-level percentile could still leave the scorer blind — are answered correctly under $\hat{w}$. Among the items that the default window breaks and $\hat{w}$ does not recover, three of seven run to the decode cap without ever emitting an answer marker, against none of the eleven items it does recover, and six of the seven sit at the middle gold position. The errors that remain are near-copies of the gold span (a dropped morpheme, an interchanged grapheme) or answers whose supporting sentence did not survive eviction — ordinary compression damage, not blindness.

K A THIRD FAMILY: LLAMA-3.1-8B-INSTRUCT

Both inputs to the rule are re-measured on the Llama tokenizer, which is costlier than Qwen's on both: the trailing blocks are $25/125/190$ tokens for English, Bengali and Telugu (Qwen: $25/107/167$) and the held-out question percentiles are Q_{90}=$18/87/94$ (Qwen: $18/76/80$), giving $\hat{w} = 43/212/284$. Baselines are $97/76/51$ (Table 11).

Bengali reproduces the hole at the default window — $76 \rightarrow 58$, -18 pp (2 fixed / 20 broken, p=.0001), with a 95% interval of $[-21.5, -9.2]$; the full-pool Telugu hole ($[-22.4, -17.3]$) is the other interval-certified hole in the paper. Here $\hat{w}$=212 recovers $+10$ pp (13/3, p=.021) while remaining -8 pp below baseline (p=.057). English is at ceiling ($97/98/98$). Telugu shows no clear hole at the default (-5, ns) and $\hat{w}$ meets the gate (-1), but the cell is low-confidence: 88 of 100 Telugu outputs never emit the answer marker, so scoring rests entirely on the first-sentence rule and the marker-only robustness check that backs the Qwen tables does not exist here. We quote Telugu-Llama only as "no clear hole; the formula does not hurt".

The residual audit is more mixed than at 8B. Of the eleven items that the baseline answers and $\hat{w}$ does not, ten have questions within Q_{90} (the window covers them fully — 91% against 88% in the pool, Fisher p=1.0; seven of the eleven sit at the middle gold position, and 4 of 10 unrecovered default-window breaks ramble to the decode cap), while one belongs to the long decile — the first arm in which the registered percentile leftover is actually visible: 10 of 12 long-question items are correct under $\hat{w}$, against 12 of 12 at 8B.

Table 11: Llama-3.1-8B-Instruct, n=100 per cell, eviction ratio 0.75, own stack. Accuracy, with the paired difference against this model's uncompressed baseline and its exact 95% interval.

	c	$\hat{w}$	baseline	w=64	at $\hat{w}$
en	25	43	97	98 ($+1$)	**98** ($+1$; $[-0.9, +1.0]$)
bn	125	212	76	58 (-18^*)	**68** (-8; $[-12.7, +0.2]$)
te	190	284	51	46 (-5)	**50** (-1; $[-5.6, +4.4]$)

L TWO MORE SLICES: QWEN3-32B, AND A 16K PREFILL

Both arms were preregistered against the fifth review and run on fresh pods; each store is self-contained with its own baselines.

Qwen3-32B (n=100 per language, own stack). The shipped integers reproduce on the 32B checkpoint (shared tokenizer and template; the driver re-derived c=$25/107/167$ and Q_{90}=$76/80$ on pod). Bengali: baseline 80, the default window certifies a -12.0 pp hole (1/13, $[-13.9, -4.5]$), $\hat{w}$=183 misses the point gate at -5.0 ($[-8.5, +1.8]$) with a $+7.0$ recovery that does not reach significance (p=.065) — the Bengali residual at every scale above 4B, now a third time. Telugu: baseline 56, and n=100 cannot distinguish a hole from noise (-4.0, 8/12, $[-12.4, +5.6]$); $\hat{w}$ meets the point gate (-1.0, wide). We registered "caution, not victory" and got it: scale softens every magnitude, and the honest sentence is that the 32B Telugu cell is inconclusive at this power. One scoring caveat, the Llama-Telugu precedent again: 32B answers in prose and then copies the instruction's placeholder verbatim after the marker, so marker compliance is 0–1% and the marker-only robustness check does not exist for this arm; every 32B number rides the registered lenient scorer's prose branch.

16k prefill (Telugu, n=100, the same first-100 qids). The blind mode is prefill-independent, as the threshold account requires: at 16k the default window loses -15.0 (3/18, $[-19.7, -5.7]$, p=.0015) from a baseline of 54. The SAME $\hat{w}$=247 — no re-measurement, both inputs are prefill-free — meets the gate (-2.0, $[-6.6, +4.1]$) and recovers $+13.0$ (15/2, $[+4.6, +16.5]$, p=.0023). The 8k-vs-16k magnitudes (-19 to -20 vs -15) are stated descriptively; the item sets differ in distractor packing, so no cross-length test is run.

M ONE ENGLISH INSTRUCTION FOR EVERY LANGUAGE

Every language-dependent number in the main text comes from translated instructions. Many deployments instead ship one English instruction to all languages, which sets $c \approx 25$ for every language and removes the blind mode by construction — but not partial visibility: at w=64 with c=25 the slack is 39 tokens, below the held-out question medians of Bengali (46) and Telugu (54), so most items still show the scorer only part of their question. A preregistered arm ran this cell: English instruction, Bengali and Telugu questions and passages, instruction-last, its own baselines, n=100 per language, with c re-measured for the cross prompt (25) and the rule's prescription $\hat{w} = 25 + Q_{90}$ alongside.

Table 12: English instruction, non-English questions. Accuracy, with paired Δpp against the arm's own uncompressed baselines and exact 95% intervals; n=100; Qwen3-4B, ratio 0.75. No difference is significant.

	baseline	w=64	$\hat{w}$
bn ($\hat{w}$=101)	79	76 (-3; $[-7.7, +3.6]$)	77 (-2; $[-6.6, +4.1]$)
te ($\hat{w}$=105)	62	58 (-4; $[-7.5, +2.4]$)	58 (-4; $[-4.0, +0.8]$)

Neither language shows a significant loss at the default window, and the registered enrichment test has nothing to find: 70 (bn) and 93 (te) of 100 items have $V < 1$ at w=64, the cells break 6 items each, and the V<1 share among broken items matches the pool (Fisher p=.67 and 1.0). This is the registered scope-narrowing branch: partial visibility with an English-sized tail is benign on these items — consistent with English at w=32 and with Gemma's undamaged blind cells — and the damage the paper documents requires a trailing block long enough to hide the question *entirely*. The n=100 intervals cannot rule out losses near 7 pp, and the prompt family here is new (English instruction, non-English passages), so these six cells are never pooled with the frozen-instruction stores. One robustness note: under the stricter marker-only scorer the Telugu cells lose 12–15 pp — compression damages the #### formatting convention in this cross-lingual prompt while the answer content largely survives, a formatting fragility we report here and do not carry into any headline number.

N RIVAL REMEDIES, AND A SECOND MEMBER OF THE FAMILY

Two remedies compete with sizing the window: switch to a scorer that never reads the question, so that nothing can be hidden from it, or ship one larger constant and stop measuring. Both were run on the same items, at matched ratios, against each store's own uncompressed baseline (Table 13). A third block asks the complementary question: whether the integer we compute is a property of the prompt or of the method it was first tested on.

Scorers that do not read the question. Expected Attention ranks by an estimate of future attention (Devoto et al., 2025) and TOVA by attention already paid (Oren et al., 2024); neither can be blinded by a trailing block, because neither was looking. At r=0.75 both lose significantly in every language — including English, where the trailing block is 25 tokens and no window we tested costs anything. Their damage does not follow c, which is what distinguishes it from the failure this paper describes; it is simply the price of ranking a cache without knowing what is being asked. Blindness is a property of a window that can be sized; query-agnosticism is a property of the method.

One prediction registered for this arm cannot be scored as written: it compares Thai against Bengali, and the arm as locked runs English, Bengali and Telugu. We flag the drafting error rather than substitute a different comparison after the fact. The claim it was meant to test — that this damage does not track c — is carried by the English column, which was written into the same preregistration.

One larger constant. At r=0.9375 the retained cache is about 512 tokens on these 8k prompts, so a w=256 window protects half of what survives. It costs English 9 pp against baseline (p=.049; against the default window the same gap is -7 and does not reach significance at n=100), while $\hat{w}_{\text{en}}$=43 costs one. It does not beat $\hat{w}$ anywhere: -8 pp on English (p=.077) and -4 pp on Bengali (ns), so the dominance is carried by English and we do not claim it per language. Bengali collapses

at the default window here (-44 pp) and $\hat{w}$ recovers $+20$ pp of that (22 fixed / 2 broken, $p<10^{-4}$), its largest recovery at this ratio — and still finishes 24 pp below baseline. Sizing the window is what removes the blind mode; it is not what makes aggressive eviction free.

The reviewer-facing question is the other ratio: does the constant suffice where the headline numbers live? Measured on its own pod and baselines, judged by the same preregistered ± 3 gate the rule was judged by, w=256 at r=0.75 is $+1$ on English, -3 on Telugu — at the boundary — and -5 on Bengali (5 fixed / 10 broken, ns, CI $[-11.5, +3.5]$), missing the gate that $\hat{w}$ met at -2. The preregistration expected the constant to pass and committed to rewriting our claim if it did; it did not, and we state the symmetric caveat instead: at n=100 the Bengali interval certifies neither the miss nor a close, so what the grid supports is this: at the headline ratio, on its own pod and baselines, the constant meets the gate on English, sits at the boundary on Telugu, and misses it on Bengali — the gate the rule met on the campaign pod; at r=0.9375, where the two run on one pod, the constant trails the rule on both languages (-9 vs -1 on English, -28 vs -24 on Bengali) and beats it nowhere they share a pod.

The constant at scale. The grid above is bounded by its pod and its hundred items, so we preregistered a re-run at the full validation pools (`iclr-constant-depth-preregister.md`; registered and externally timestamped before any row existed): w=256 beside the shipped $\hat{w}$ integers, on the campaign stack, with baselines byte-identical to the depth arm's (782/782) — so unlike the bracketed cells above, every comparison here pairs within one stack. The registration fixed the Telugu reading in advance: the constant sits nine tokens above the computed integer ($256 = \hat{w}_{\text{te}}+9$), so no separation was expected there, and none was found — the head-to-head is $+1.2$ pp in $\hat{w}$'s favour (16 fixed / 8 broken, CI $[-0.4, +2.5]$): a tie, with zero inside the interval by four tenths of a point. Where the two windows genuinely differ, Bengali, the point gate splits: $\hat{w}$ meets it (-1.8) and the constant misses it (-3.5, CI $[-9.9, +4.1]$), with an uncertified head-to-head ($+1.8$, CI $[-3.6, +5.9]$) on a pool its size caps at 113. Against its own baseline the constant's Telugu residual is certified at -6.9 ($[-9.0, -4.2]$), deeper than $\hat{w}$'s -5.7: a 256-token protected tail costs slightly more than a 247-token one and recovers nothing. We claim no certified separation at this ratio. What the full pools support: the constant misses both of its point gates where the rule misses one, its point estimate trails the rule's in both languages, and it comes closest exactly where it reproduces the rule's own integer plus nine.

No ranking at all. The same store carries the control the registry always reserved: `random@r0.75`, eviction with no scorer. It does not lose answers; it collapses generation — $6/1/0\%$ accuracy on English, Bengali and Telugu, with 96–100% of outputs running to the decode cap, mostly as repetition loops. Even a blinded ranking is carrying enormous weight. Because random sits at the floor for every language, it bounds the value of ranking from above without discriminating between them, and we draw no per-language conclusion from it.

A second member of the family. PyramidKV (Cai et al., 2024) keeps the observation window and changes what is done with the retained budget, allocating more of it to lower layers; its kvpress implementation subclasses SnapKV and inherits w=64 unchanged. On its own pod and its own baselines, that inherited default digs holes of -28 and -32 pp — deeper than the -16 and -19 the campaign pod recorded for SnapKV, a cross-pod contrast we quote as context, not as a test — and the AutoWindow integers, used exactly as shipped, remove them: -2, -1 and $+0$ on Bengali, Telugu and English, with recoveries of $+26$ (29 fixed / 3 broken) and $+31$ (32/1), the two largest in this paper. We registered the opposite guess about the depth, expecting the pyramid allocation to shallow the blind hole rather than deepen it, and report the outcome as an observation we did not predict. The transfer itself is the point: an integer computed from a tokenizer and a template, never tuned against accuracy on either method, repairs both.

O A SECOND CONSTANT: THE DECODE CAP

The main arms decode under a 384-token cap, and Appendix Q says why in a sentence; this appendix is the measurement behind it. The campaign's first arms used 128, and a decode cap is itself a token-denominated constant — so the paper's thesis applies to it, one layer further out, in the evaluation harness rather than in the evictor. The evidence comes from the cap-128-era stores: a different stack

Table 13: Rival remedies, paired Δpp against each store's own uncompressed baseline, n=100 per cell; star: McNemar $p < .05$. First block: two query-agnostic scorers at the ratio used throughout the paper, beside the windowed cells they would replace. Second: the heavy-ratio sweep, where $\hat{w}$ is 43 for English and 183 for Bengali. Third: the constant and the no-ranking control at the headline ratio. Fourth: PyramidKV. Both ran on pods that did not reproduce the campaign stack, so their bracketed SnapKV values are context rather than comparators and are never pooled with them.

r=0.75	Exp. Attention	TOVA	SnapKV w=64	SnapKV $\hat{w}$
en	-14^*	-8^*	$+0$	$\mathbf{+2}$
bn	-18^*	-16^*	-16^*	$\mathbf{-2}$
te	-20^*	-18^*	-19^*	$\mathbf{+0}$
r=0.9375	w=64	w=256	$\hat{w}$	
en	-2	-9^*	$\mathbf{-1}$	
bn	-44^*	-28^*	$\mathbf{-24}^*$	
r=0.75, own pod	random	w=256	[SnapKV $\hat{w}$]	
en	-87^*	$\mathbf{+1}$	[+2]	
bn	-72^*	-5	[−2]	
te	-56^*	$\mathbf{-3}$	[+0]	
PyramidKV	w=64	$\hat{w}$	[SnapKV: w=64 / $\hat{w}$]	
en	$+2$	$\mathbf{+0}$	[+0 / +2]	
bn	-28^*	$\mathbf{-2}$	[-16^* / −2]	
te	-32^*	$\mathbf{-1}$	[-19^* / +0]	

and the byte-parallel item variant, whose inputs are byte-equal across languages. Nothing here is pooled with the main tables, and every number is recomputed by `decode_cap_ledger.py`.

At a 128-token cap, the share of *uncompressed* baseline generations that hit the cap runs from 1% in Chinese to 65% in Greek (Table 14) — a spread produced by one integer, on byte-equal inputs, with no compression anywhere in the picture. Because the instruction asks for the answer marker at the end of the reply, truncation does not merely shorten an answer: it decides which branch of the scoring rule fires. At 128, 95% of English baselines carry the marker against 45% of Greek; at 384 the marker comes back (90/97/92% for Greek, Hindi and Russian). An English-validated harness would see none of this.

Greedy decoding makes the 128-token output a byte prefix of the 384-token output on 1,124 of the 1,200 generations the two runs share, so the cap's effect is isolable within an item rather than inferred across conditions. On the 327 generations where truncation destroys the marker, our scoring rule loses 4 pp (19 fixed / 7 broken, p=.029; Greek +2 and Hindi +1, both ns, Russian +11, p=.039). That is real and it is bounded, because the rule's prose-fallback branch was built to absorb exactly this failure. We cannot say from these stores what a format-strict harness would lose — our own strict marker path also fails for reformatting reasons that have nothing to do with the cap — so we claim only the part we can measure: the cap selects the scoring branch per language, and under our rule the residual cost is four points on the cells it touches. This is why the main arms decode at 384, and why we treat every token-denominated constant in the pipeline — window, cap, budget — as a quantity to measure rather than a default to inherit.

P FROZEN INSTRUCTIONS AND PROMPT LAYOUT

The QA instructions are one line per language, frozen under a prompt version key so that a change to them would invalidate the stored runs rather than silently alter them. They carry the same communicative content — answer from the passages above, end with a marked span — and differ only in how many tokens the model's tokenizer needs for it: 20 for English, 102 for Bengali, 162 for Telugu on Qwen3-4B. Every instruction-last item has the layout

```
[gold + distractor passages, packed to 8k tokens]
<question>
```

Table 14: The decode cap as a second token-denominated constant. Uncompressed baselines on byte-parallel items, n=100 per language, from the cap-128-era stores; Greek, Hindi and Russian were re-run at 384. Truncated: share of generations that reach the cap. Marker: share carrying the end-of-reply answer marker that the scoring rule keys on.

	el	hi	ru	th	de	en	vi	es	zh
truncated at 128	65%	50%	30%	14%	12%	6%	4%	4%	1%
marker at 128	45%	59%	71%	58%	74%	95%	94%	96%	98%
truncated at 384	17%	4%	2%	—	—	—	—	—	—
marker at 384	90%	97%	92%	—	—	—	—	—	—

```
<instruction>
<chat suffix>          # Qwen: <|im_end|>\n<|im_start|>assistant\n
```

and the observation window is the last w tokens of that concatenation. In English, for instance:

```
... Paris is the capital of France. ...
What is the capital of France?
Answer the question using only the passages above.
End your reply with '#### <exact answer span>'.
<|im_end|>
<|im_start|>assistant
```

The padded conditions prepend content-free filler to the instruction so that the marked-span sentence stays adjacent to generation and only the token count changes; the schema conditions prepend a JSON response specification in the same position.

Q MEASUREMENT TRANSCRIPTS AND RUN PROVENANCE

Stacks. Each generation record stores a stack descriptor (GPU, driver, CUDA, torch, transformers, kvpress) and every comparison in this paper is within one descriptor. The Gemma cells share one stack and the Qwen3-8B slice another; the Qwen3-4B cells share a third, with one exception — the PyramidKV and constant-control pods each landed on descriptors of their own (a newer driver patch or a different host kernel; same GPU model and library versions), so those arms pair only within themselves and are quoted beside the others rather than pooled with them. We never pool across descriptors.

Determinism, measured. The follow-up runs were executed weeks after the original ones, on new machines, and they re-ran cells that the earlier stores already contained — uncompressed baselines, default-window cells, the rungs a later ladder shares with an earlier one. We do not count these by hand: `determinism_ledger.py` keys every generation by model, task, language, treatment, item and prompt version, takes the earliest run of each key as the original, and compares every later run against it. It finds 6,763 generations produced a second time in a later store, and all 6,763 reproduce the original text byte for byte — the largest single block being the slack-ablation store, whose baseline and $\hat{w}$ cells repeat all 1,338 of the depth arm's corresponding rows exactly. This is what allows us to treat the re-measured constants and the earlier tables as describing the same system. Three hundred of those pairs also cross a naming difference — one store requests the default window implicitly and the other names w=64 explicitly — and agree token for token, as they should. The drifted pods bought a measurement we had not planned: 2,138 generations are shared across stack boundaries — the PyramidKV and constant-control baselines, and now the entire baseline and $\hat{w}$ cells of the oracle arm, which reproduce the depth arm's 1,338 corresponding rows byte-for-byte from a different pod — and every one is byte-identical across the boundary it crosses. The ledger counts same-descriptor repeats (6,763, the figure above) and cross-descriptor ones (2,138) separately, and the argument that the earlier and later tables describe one system rests only on the former. For completeness: two earlier pod attempts of the final arms died partway (one to a guard abort with zero generations, whose log ships with the record; one to container loss), and the storage volume holding their partial stores — 1,106 and 409 rows, each on a stack descriptor of its own —

was deleted outside our control. Neither partial store matched the campaign descriptor, so neither was ever a candidate for analysis; the completed arms were regenerated in full on the campaign stack and are the only versions any number in this paper draws on.

Decode cap. Every arm reported in the main text uses a 384-token decode cap. The supporting window-widening comparison in Appendix R was measured under a 128-token cap, as were the historical instruction-first numbers kept beside it; the instruction-first layout swap itself has been re-measured at the current cap and is reported in the main text (Table 2). We flag the difference here rather than leave it to be discovered in a log. The cap matters, and it is an instance of this paper's subject rather than a detail of it: at 128 tokens the share of answers truncated varies from 1 to 65% across languages, which is why the main arms were re-run at 384 (Appendix O).

R TWO FURTHER LEVERS ON VISIBILITY

Two earlier interventions move the same quantity and are consistent with the account in the main text. Both were measured under the 128-token decode cap noted above, so we report them as supporting rather than as headline evidence; the layout swap has since been re-measured at the current cap and moved to the main text. The earlier cap-128 measurement is retained here only as provenance: Bengali $+14$ pp at a fixed ratio (34/6, $p<10^{-5}$, $n=200$) and $+22$ pp under an absolute cache budget (51/7, $p<10^{-8}$, $n=200$), with a Thai cost of 6.5 pp. The current-cap rerun locates that Thai cost in the uncompressed baseline (a layout effect, -5 pp, ns) rather than in compression, and notes that Thai drops the answer marker in 43% of question-last outputs, so part of the shift is format behaviour. Widening the window from 64 to 128 tokens at a fixed ratio recovers Bengali by 10.5 pp (22 fixed / 1 broken, $p<10^{-5}$, $n=200$) and moves no further in these pairings: 256 tokens adds nothing detectable (-0.5 pp, 8/9). The dose ladder, at the current cap and $n=100$, still gains from slack 16 to 32, so we read this cap-128 pairing as supporting the coarse shape only.

S THE LAYOUT IN THE WILD

Our items are constructed, which invites a fair question: does any widely used prompt actually put a long block after the question? We surveyed 22 templates selected by popularity and officialness *before* their bodies were read — the default QA prompt of a framework above 5k stars, an official vendor documentation example, or the prompt of a published benchmark or harness — so that safe layouts could not be filtered out. Each was recorded verbatim from a commit-pinned source, the list was frozen, and only then was anything tokenized. We measure the *static* trailing block: the text after the question placeholder with every unfilled placeholder deleted, on the Qwen3-4B tokenizer. Deleting placeholders makes every number a lower bound, because runtime content only adds (Table 15).

The survey measures geometry; for its strongest single-shot example the damage is now measured too. A preregistered arm rendered the shipped refine template (T06) verbatim over our items — query first, a fixed existing-answer stub, passages as the new context, so $V=0$ for every item at any small window. Total blindness in that layout costs English a non-significant -3.0 ($[-4.9, +2.2]$, $n=100$) and Bengali -8.0 ($[-12.7, +0.2]$, $p=.057$) — blindness is necessary for the failure, not sufficient (the Gemma blind-English cells again), with the Bengali interval leaving room for a real effect. AutoWindow's output for this layout is a refusal — c is the entire post-question suffix, so no practical window clears it; the remedy is to protect or reorder, not to resize. Whether eviction harms an agent LOOP mid-trajectory remains untested — the question at step k is not the task at step 0 — and the T22-style templates stay static measurements.

Most of what ships is safe, and that is the first half of the result. Single-shot retrieval-QA defaults leave 4–5 tokens after the question, and their chat variants leave 0–4. Modern vendor structured-output and tool-use APIs carry the schema in an API field rather than in prompt text, so at most a token handful (≤ 6) remains after the task — which falsifies the guess we began with, that in-prompt response schemas would be where long trailing blocks live. Harness QA templates leave 3–5 tokens, bar one at 19/15 (T22).

The blocks appear where a prompt is *iterated*. LlamaIndex's shipped refine template puts the query first and, at its empty-placeholder floor, exactly 64 tokens after it — the kvpress default, to the token,

before the existing answer or the new context contributes anything. SWE-agent's default instance template puts 221 static tokens after the problem statement, longer than the longest trailing block among our eight languages, before its first observation arrives. ReAct and the OpenHands loop render the user's question once and append every later tool result after it by construction. The full record, with permalinks and verbatim template text, ships with the code.

Table 15: Static trailing block of 22 popular templates, in tokens on the Qwen3-4B tokenizer, with unfilled placeholders deleted so each number is a lower bound. Zero means the question is last. Selection was frozen before any template body was read or tokenized.

	template	tokens
Single-shot retrieval QA		
T01	LangChain `RetrievalQA`, completion	5
T02	LangChain `RetrievalQA`, chat	0
T03	LangChain hub `retrieval-qa-chat`	0
T04	LlamaIndex text-QA default	4
T05	LlamaIndex text-QA, chat	4
T08	Haystack `PromptBuilder` example	5
T09	RAGFlow chat assembly	0
Structured output and tool use		
T10	OpenAI structured outputs	0
T11	OpenAI function calling	0
T12	OpenAI JSON mode example	6
T13	Anthropic tool use	0
T14–T16	Instructor, Outlines, Guidance	0
Evaluation harnesses		
T20	`lm-eval` XQuAD, English	3
T21	`lm-eval` XQuAD, Arabic	5
T22	Belebele, official / harness	19 / 15
Iterated prompts		
T07	LlamaIndex refine, chat	9
T17	LangChain ReAct, static suffix	3
T06	**LlamaIndex refine default**	**64**
T18	**SWE-agent instance template**	**221**
T19	OpenHands mid-loop	grows

T PRODUCTION SCOPE

Re-checked on 2026-08-18, with permalinks in the supplementary record. vLLM exposes KV-cache dtype and scheduler token budgets, not a SnapKV-style evictor; its default attention backends are dense. TensorRT-LLM implements last-w scoring as an opt-in sparse-attention backend whose configuration default is a 32-token window, and skips it entirely below a prompt-length threshold. SGLang has an open proposal for a sparsity framework which states that it must be default-off and names SnapKV and PyramidKV among its explicit non-goals — so the family we study is, for now, deliberately outside the default path of the two most used open serving stacks. Mao et al. (2026) argue that research evictors rarely ship as production defaults, and we cite that against ourselves: the constant we study is a research default and one opt-in production constant, not something most deployments are running today. The reason to fix it now is that the fix is a measurement rather than a redesign.